P-NP Problems

Tractable Vs Intractable

- We have so far studied algorithms whose complexities were polynomial in n where n is a suitably defined input size.
  - These are "tractable problems"- not so hard
- Here we focus on problems that have algorithms with exponential complexity.
  - Even best known algorithms may take years or centuries on fastest computers.
  - These are "intractable problems" – hard.
- Decision Problems
  - We generally face optimization problems
  - Shortest path, knapsack, matrix-chain etc.,...
- NP-Completeness restricts attention to decision problems
  - They have either **yes** or **no** as the solution
  - But have to provide an additional input to specify a problem instance

Decision Problems

- To keep things simple, we will mainly concern ourselves with decision problems. These problems only require a single bit output: “yes” and “no”.

How would you solve the following decision problems?

- Is this directed graph acyclic?
- Is there a spanning tree of this undirected graph with total weight less than w?
- Does this bipartite graph have a perfect (all nodes matched) matching?
- Does the pattern P appear as a substring in text T?

All the decision problems mentioned above are in P.

P and NP

What is P?

- P is the set of all decision problems which can be **solved** in polynomial time by a deterministic Turing machine. Since it can be solved in polynomial time, it can also be verified in polynomial time.

What is NP?

- NP is the set of all decision problems (question with yes-or-no answer) for which the 'yes'-answers can be **verified** in polynomial time ($O(n^k)$ where n is the problem size, and k is a constant) by a deterministic Turing machine.
- Polynomial time is sometimes used as the definition of *fast* or *quickly*.
- Only a non-deterministic computer solves the NP problem in a polynomial time. Therefore P is a subset of NP.

Non-Determinism

- Think of a non-deterministic computer as a computer that magically "guesses" a solution, then has to verify that it is correct
  - If a solution exists, computer always guesses it
  - One way to imagine it: a parallel computer that can freely spawn an infinite number of processes
    - Have one processor work on each possible solution
    - All processors attempt to verify that their solution works
    - If a processor finds it has a working solution
  - So: **NP** = problems *verifiable* in polynomial time

NP

- The class NP (meaning Non-deterministic Polynomial time)
- What makes these problems special is that they might be hard to solve, but a short answer can always be printed in the back, and it is easy to see that the answer is correct once you see it.

- Example… Does the given graph have a Hamiltonian Path?
  - No guarantee if answer is “no”.
  - But given a path, can we verify if it is correct? “Yes”

P vs NP

- Some problems are *intractable*: as they grow large, we are unable to solve them in reasonable time
- What constitutes reasonable time? Standard working definition: *polynomial time*
  - On an input of size n the worst-case running time is $O(n^k)$ for some constant k
  - Polynomial time: $O(n^2)$, $O(n^3)$, $O(1)$, $O(n \lg n)$
  - Not in polynomial time: $O(2^n)$, $O(n^n)$, $O(n!)$

Polynomial-Time Algorithms

- Are some problems solvable in polynomial time?
  - Of course: every algorithm we've studied provides polynomial-time solution to some problem
  - We define **P** to be the class of problems solvable in polynomial time
- Are all problems solvable in polynomial time?
  - No: Turing's "Halting Problem" is not solvable by any computer, no matter how much time is given
  - Such problems are clearly intractable, not in **P**

```
Ex: while (true) continue
```

Tractability

- Some problems are *undecidable*: no computer can solve them
  - E.g., Turing's "Halting Problem"
  - We don't care about such problems here;

- Other problems are decidable, but *intractable*: as they grow large, we are unable to solve them in reasonable time
  - *What constitutes "reasonable time"?*

NP-Complete Problems

- The *NP-Complete* problems are an interesting class of problems whose status is unknown
  - No polynomial-time algorithm has been discovered for an NP-Complete problem
  - No suprapolynomial lower bound has been proved for any NP-Complete problem, either
- We call this the *P = NP question*
  - The biggest open problem in CS

NPC – Hamiltonian Cycles

- An example of an NP-Complete problem:
  - A *hamiltonian cycle* of an undirected graph is a simple cycle that contains every vertex
  - The hamiltonian-cycle problem: given a graph G, does it have a hamiltonian cycle?
  - *Describe a naïve algorithm for solving the hamiltonian-cycle problem. Running time?*

P and NP

- Summary so far:
  - **P** = problems that can be solved in polynomial time
  - **NP** = problems for which a solution can be verified in polynomial time
  - Unknown whether **P = NP** (most suspect not)
- Hamiltonian-cycle problem is in **NP**:
  - Cannot solve in polynomial time
  - Easy to verify solution in polynomial time (*How?*)

Is P = NP?

- **P** = set of problems that can be solved in polynomial time
- **NP** = set of problems for which a solution can be verified in polynomial time
- $\mathbf{P} \subseteq \mathbf{NP}$
- The big question: Does **P = NP**?

NP Complete Problems

- We will see that NP-Complete problems are the "hardest" problems in NP:
  - If any *one* NP-Complete problem can be solved in polynomial time…
  - …then *every* NP-Complete problem can be solved in polynomial time…
  - …and in fact *every* problem in **NP** can be solved in polynomial time (which would show **P = NP**)
  - Thus: solve Hamiltonian-cycle in $O(n^{100})$ time, you've proved that **P = NP**.

Reduction

- The crux of NP-Completeness is *reducibility*
  - Informally, a problem P can be reduced to another problem Q if *any* instance of P can be "easily rephrased" as an instance of Q, the solution to which provides a solution to the instance of P
    - *What do you suppose "easily" means?*
    - This rephrasing is called *transformation*
  - Intuitively: If P reduces to Q, P is "no harder to solve" than Q

Reducibility

- An example:
  - P: Given a set of Booleans, is at least one TRUE?
  - Q: Given a set of integers, is their sum positive?
  - Transformation: $(x_1, x_2, \ldots, x_n) = (y_1, y_2, \ldots, y_n)$ where y_i = 1 if x_i = TRUE, y_i = 0 if x_i = FALSE
- Another example:
  - Solving linear equations is reducible to solving quadratic equations

Using Reductions

- If P is *polynomial-time reducible* to Q, we denote this $P \leq_p Q$
- Definition of NP-Complete:
  - If P is NP-Complete, $P \in$ **NP** and all problems R are reducible to P
  - Formally: $R \leq_p P \; \forall \; R \in$ **NP**
- If $P \leq_p Q$ and P is NP-Complete, Q is also NP-Complete

NP-Hard and NP-Complete

- If P is *polynomial-time reducible* to Q, we denote this $P \leq_p Q$
- Definition of NP-Hard and NP-Complete:
  - If all problems $R \in$ **NP** are reducible to P, then P is *NP-Hard*
  - We say P is *NP-Complete* if P is NP-Hard and $P \in$ **NP**

- If $P \leq_p Q$ and P is NP-Complete, Q is also NP- Complete

NP Complete Problems

- Given one NP-Complete problem, we can prove many interesting problems NP-Complete
  - Graph coloring (= register allocation)
  - Hamiltonian cycle
  - Hamiltonian path
  - Knapsack problem
  - Traveling salesman
  - Job scheduling with penalities
  - Many, many more

Why Prove NP-Completeness?

- Though nobody has proven that **P** != **NP**, if you prove a problem NP-Complete, most people accept that it is probably intractable
- Therefore it can be important to prove that a problem is NP-Complete
  - Don't need to come up with an efficient algorithm
  - Can instead work on *approximation algorithms*

Proving NP-Completeness

- *What steps do we have to take to prove a problem P is NP-Complete?*
  - Pick a known NP-Complete problem Q
  - Reduce Q to P
    - Describe a transformation that maps instances of Q to instances of P, s.t. "yes" for P = "yes" for Q
    - Prove the transformation works
    - Prove it runs in polynomial time

NP-Complete – Example: Clique Problem

- A clique is a complete subgraph (each pair of vertices is connected by an edge)
  - Size of a clique is the number of its vertices
- The clique problem
  - **Optimization problem**: Given a graph G=(V,E), find the clique of maximum size
  - **Decision problem**: Given a graph G=(V,E) and an integer k, is there a clique of size k?
- Consider the decision problem
  - Naïve approach: list all k-subsets of V and check each to see if it forms a clique
  - Complexity proportional to n^k, where n=|V|, but k is not a constant
- An efficient algorithm is unlikely to exist
  - No one has found one, but…
  - No one has proved no such algorithm exists
- The clique problem is **NP**-complete !

Example: Satisfiability Problem (SAT)

- Given a Boolean formula (with variables and Boolean operators AND, OR and NOT), is it satisfiable?
  - Is there an assignment of values 0 or 1 to variables so that the formula evaluates to 1?
- Conjunctive Normal Form (CNF)
  - A clause is the OR of some literals
  - A Boolean formula consisting of several clauses separated by AND is a CNF formula
    - E.g., (a+b+c)(b'+d+e'+f)(a'+e)
- 3-CNF: a CNF formula in which each clause has 3 literals
  - E.g., (a+b+c')(d+e'+f)(a'+f'+g)
- Given a 3-CNF formula, is it satisfiable?
  - That is, is there an assignment (to variables) that evaluates the formula to 1
  - This is also called the 3-CNF-SAT problem.

Few other examples…

Graph Coloring Problem

- Given a graph, how to color vertices so that adjacent ones have different colors
  - Chromatic number is the smallest number of colors needed to color a graph
  - **Optimization problem**: Given a graph G=(V,E), find the chromatic number
  - **Decision problem**: Given a graph G=(V,E) and an integer k, is G k-colorable?
- Subset Sum Problem
- Given a set S of positive integers and an integer k
- Is there a subset R of S such that the sum of the elements in R is equal to k?
- Example
  - S={1,16,64,256,1040,1041,1093,1284,1344} and k=3754
  - R={1,16,64,256,1040,1093,1284} is a solution.

Few other examples…

Hamiltonian Path Problem

- A Hamiltonian path of a graph
  - A simple path that passes through every vertex exactly once
- Does a given undirected graph has a Hamiltonian path?
- Can also specify for directed graphs
- Travelling Salesman Problem
- Known as TSP or minimum tour problem
- A salesperson wants to minimize total traveling cost (distance or time) required to visit a set of cities and return to the starting point
- Given a weighted, complete graph and an integer k, is there a Hamiltonian cycle with total weight at most k?

Classes of P and NP

- An algorithm is polynomially bounded if its worst-case complexity is bounded by a polynomial of the input size
- Polynomially bounded problem: one that has a polynomially bounded algorithm
- **P** is the class of decision problems that are polynomially bounded
- For any of the example decision problems discussed, one may have a "proposed solution" that can be checked
- **NP** is the class of decision problems for which a given proposed solution for a given input can be checked in polynomial time to see if it really is a solution.
- Examples
  - Clique, graph coloring, Hamiltonian path, subset sum, satisfiability, TSP

Relationship between P and NP

It is not known whether **P**=**NP** or whether **P** is a proper subset of **NP**

- It is believed **NP** is much larger than **P**
  - But no problem in **NP** has been proved as not in **P**
  - No known deterministic algorithms that are polynomially bounded for many problems in **NP**
  - So, "does **P** = **NP**?" is still an open question!
- **NP**-Completeness
- "**NP**-complete problems": the hardest problems in **NP**
- Interesting property
  - If any one **NP**-complete problem can be solved in polynomial time, then every problem in **NP** can also be solved similarly
- E.g.,: satisfiability, clique, graph coloring, Hamiltonian path, subset sum, TSP.

How to prove problem is NP-Complete?

1. Show **S** is in **NP**
2. Select a known **NP**-complete problem **R**
   - Since **R** is **NP**-complete, all problems in **NP** are reducible to **R**
3. Show how **R** can be poly. reducible to **S**
   - Then all problems in **NP** can be poly. reducible to **S** (because polynomial reduction is transitive)
4. Therefore **S** is **NP**-complete

NP-Complete

What is NP-Complete?

- A problem x that is in NP is also in NP-Complete if and only if every other problem in NP can be quickly (ie. in polynomial time) transformed into x. In other words:
- x is in NP, and
- Every problem in NP is reducible to x
- So what makes NP-Complete so interesting is that if any one of the NP-Complete problems was to be solved quickly then all NP problems can be solved quickly

“NP-Complete” comes from:

- **Nondeterministic Polynomial**
- **Complete - “Solve one, Solve them all”**

NP-Hard

What is NP-Hard?

- NP-Hard are problems that are at least as hard as the hardest problems in NP.
- A problem is NP hard if every problem in NP can be polynomially reduced to it
- Note that NP-Complete problems are also NP-hard.
- However not all NP-hard problems are NP (or even a decision problem), despite having 'NP' as a prefix. That is the NP in NP-hard does not mean 'non-deterministic polynomial time’.
- All optimization problems in NP are NP-Hard and all decision problems in NP are NP-Complete

co-NP

What is co-NP?

- A decision problem X is a member of co-NP if and only if its complement X is in the complexity class **NP**.
- In simple terms, co-NP is the class of problems for which there is a polynomial-time algorithm that can verify "*no*" instances (sometimes called counterexamples) given the appropriate certificate.
- Equivalently, co-NP is the set of decision problems where the "no" instances can be accepted in polynomial time by a non-deterministic Turing machine.

Relationship between NP-Complete and NP-Hard